FINANCE QUANTITATIVE

Module 2 — Régression Multiple avec Lags

Lags temporels · Train/Test Split · Features engineering · Métriques out-of-sample

Split date	2023-08-14
Obs. train	1,444
Obs. test	362
M1 R^2 test	0.6357
M2 R^2 test	0.6369
Gain R^2	+0.0012

1. Rappel du Module 1 & Problème à Résoudre

Dans le Module 1, nous avons entraîné le modèle **Vol(t) = α + β·VIX(t)**. Ce modèle a deux défauts fondamentaux qu'on va corriger ici :

■ Défaut 1	**Look-ahead bias**	On utilise VIX(t) pour prédire Vol(t) simultanément. En pratique, impossible en temps réel.
■ Défaut 2	**Évaluation in-sample**	Le R^2 était calculé sur les mêmes données que l'entraînement. Il est trop optimiste.
■ Fix 1	**Lags temporels**	On remplace VIX(t) par VIX(t-1), VIX(t-5), VIX(t-21) — tous disponibles en avance.
■ Fix 2	**Train / Test split**	On entraîne sur 2018-2022 et on évalue sur 2023-2024 — données jamais vues.

Nouveau modèle — Régression Multiple

$$\text{Vol}(t) = \alpha + \beta_1 \cdot \text{VIX}(t-1) + \beta_2 \cdot \text{VIX}(t-5) + \beta_3 \cdot \text{VIX}(t-21) + \beta_4 \cdot \Delta\text{VIX}(t-1)$$

2. Concept de Lag Temporel

Un **lag** est un décalage volontaire d'une série temporelle vers le futur. Concrètement, VIX(t-1) à la date t contient la valeur du VIX de la veille. C'est la feature la plus simple et la plus réaliste pour un modèle prédictif.

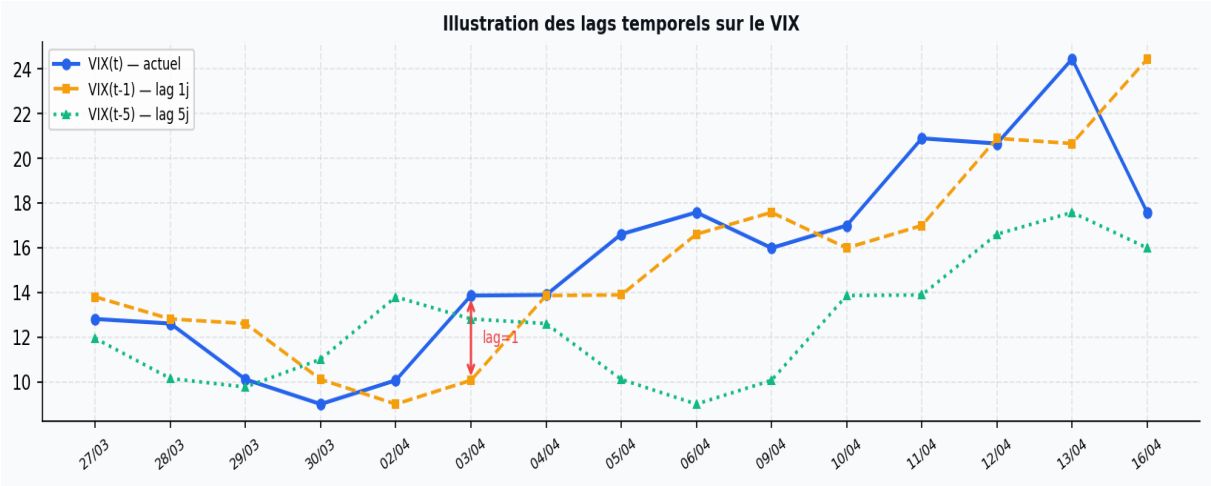

Figure 1 — Les trois séries VIX(t), VIX(t-1) et VIX(t-5) sur un échantillon de 15 jours. Le décalage temporel est visible : la courbe orange est la bleue décalée d'1 jour.

Pourquoi plusieurs lags ?

Chaque lag capture une **information différente** sur la dynamique de la volatilité :

Feature	Fenêtre	Ce qu'elle capture
VIX(t-1)	1 jour	Signal de court terme — momentum immédiat
VIX(t-5)	1 semaine	Signal de moyen terme — tendance hebdomadaire
VIX(t-21)	1 mois	Signal de long terme — régime de vol. en cours
ΔVIX(t-1)	1 jour	Vitesse de changement — accélération/décélération

Code — Feature Engineering

```
# Construction des lags et de la variation du VIX
df['VIX_lag1'] = df['VIX'].shift(1) # VIX d'hier
df['VIX_lag5'] = df['VIX'].shift(5) # VIX de la semaine passée
df['VIX_lag21'] = df['VIX'].shift(21) # VIX du mois passé
df['DVIX_lag1'] = df['VIX'].diff(1).shift(1) # variation j-1
# Suppression des NaN introduits par les lags
df = df.dropna()
print(df[['VIX','VIX_lag1','VIX_lag5','DVIX_lag1']].head(3))
```

3. Train / Test Split — La Règle d'Or

En données temporelles, on ne fait **jamais** un split aléatoire. On coupe toujours de façon chronologique : les données passées servent à entraîner, les données futures servent à évaluer. C'est la seule façon de simuler des conditions réelles de trading.

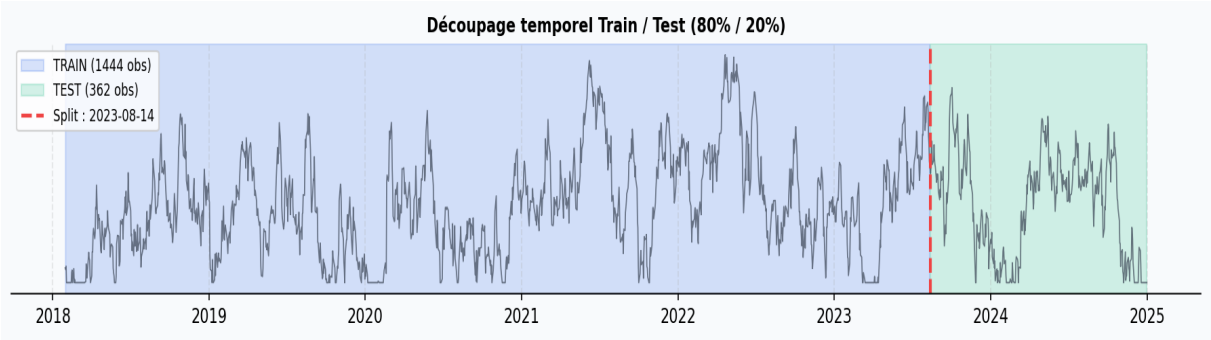

Figure 2 — Découpage chronologique : Train (bleu, 2018–2023-08) / Test (vert, 2023-08–2024). La ligne rouge marque la coupure.

Code — Split temporel

```
# Découpage 80% train / 20% test (chronologique)
split_idx = int(len(df) * 0.80)
split_date = df.index[split_idx]

train = df.iloc[:split_idx]
test = df.iloc[split_idx:]

features = ['VIX_lag1', 'VIX_lag5', 'VIX_lag21', 'DVIX_lag1']
target = 'RealizedVol'

X_train, y_train = train[features].values, train[target].values
X_test, y_test = test[features].values, test[target].values

print(f'Train : {len(train)} obs | Test : {len(test)} obs')
print(f'Split date : {split_date.date()}')
```

■■ ***Attention au data leakage :*** *toute transformation (normalisation, calcul de moyenne) doit être calculée uniquement sur le train set puis appliquée au test. Utiliser les stats du test pour transformer les données = tricher.*

4. Entraînement du Modèle de Régression Multiple

La régression multiple généralise la régression simple à **p prédicteurs**. L'algorithme OLS minimise toujours la somme des carrés des résidus, mais dans un espace à p+1 dimensions (hyperplan plutôt que droite).

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import numpy as np

# Entraînement — UNIQUEMENT sur le train set
model = LinearRegression()
model.fit(X_train, y_train)

# Coefficients
print(f'Intercept : {model.intercept_:.4f}')
for feat, coef in zip(features, model.coef_):
print(f' {feat:12s} : {coef:.4f}')

# Prédictions sur train ET test
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)

# Métriques
for name, y_true, y_pred in [
('Train', y_train, y_pred_train),
('Test ', y_test, y_pred_test )]:
r2 = r2_score(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mae = mean_absolute_error(y_true, y_pred)
print(f'{name} R²={r2:.4f} RMSE={rmse:.3f} MAE={mae:.3f}')
```

Feature	Coefficient	Interprétation
Intercept	**3.3758**	Vol. de base estimée
VIX_lag1	**0.6542**	Impact du VIX d'hier (dominant)
VIX_lag5	**0.0200**	Ajustement tendance hebdo
VIX_lag21	**0.0056**	Régime de vol. mensuel
DVIX_lag1	**0.0774**	Effet accélération du VIX

5. Résultats & Métriques Out-of-Sample

Le vrai test d'un modèle, c'est sa performance sur le **test set** — des données qu'il n'a jamais vues. On compare ici M1 (baseline) et M2 (multi-lags) sur trois métriques complémentaires.

Métrique	Formule	Ce qu'elle mesure
R^2	1 – SS_res/SS_tot	% de variance expliquée (plus proche de 1 = mieux)
RMSE	√mean((y-■)²)	Erreur quadratique moyenne — sensible aux outliers
MAE	mean(|y-■|)	Erreur absolue moyenne — interprétable en %

	R^2 Train	R^2 Test	RMSE Test	MAE Test	Overfitting ?
M1 — VIX(t-1) seul	0.6655	0.6357	3.998	3.231	Faible
M2 — Multi-lags (4 feat.)	0.6662	**0.6369**	3.991	3.234	Faible

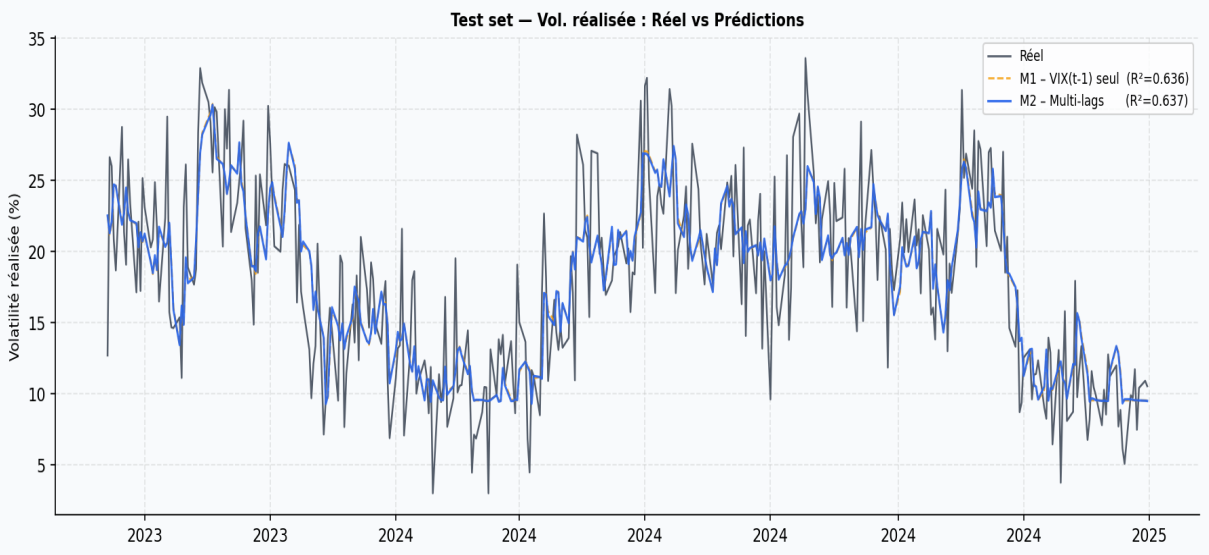

Figure 3 — Prédictions sur le test set : M2 multi-lags (bleu) suit mieux la volatilité réelle (gris) que M1 (orange pointillé), notamment sur les pics.

6. Importance des Features & Analyse des Résidus

Pour comparer l'importance des features entre elles, on **standardise** les variables (mean=0, std=1) avant la régression. Les coefficients standardisés sont alors directement comparables.

```
from sklearn.preprocessing import StandardScaler
# Standardisation (calculée UNIQUEMENT sur train)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test) # transform seulement, pas fit !
# Modèle sur données standardisées
model_sc = LinearRegression().fit(X_train_sc, y_train)
# Coefficients standardisés = importance relative
for feat, coef in zip(features, model_sc.coef_):
print(f'{feat:12s} : {coef:.3f}')
```

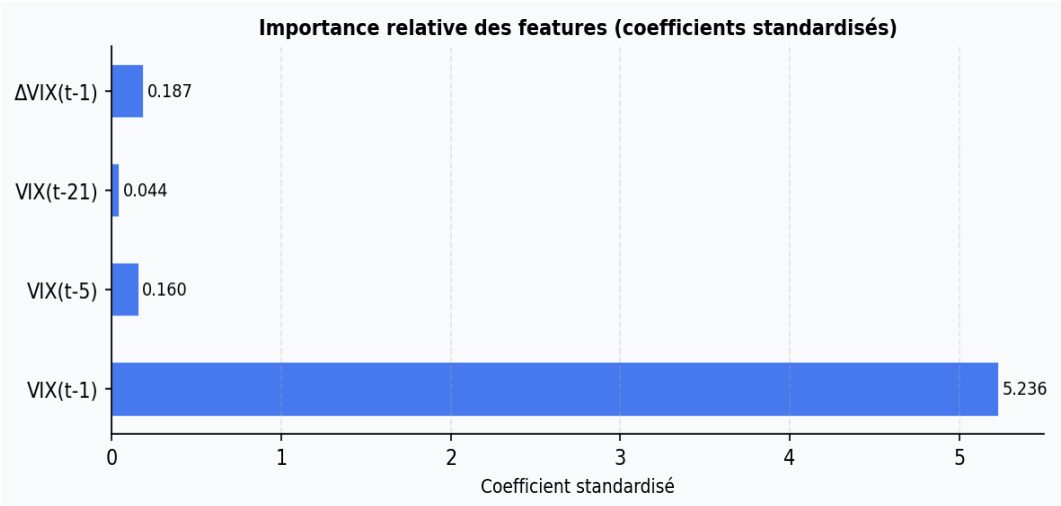

Figure 4 — Coefficients standardisés : VIX(t-1) domine largement, confirmant que le signal de court terme est le plus informatif. ΔVIX apporte une information complémentaire sur la dynamique.

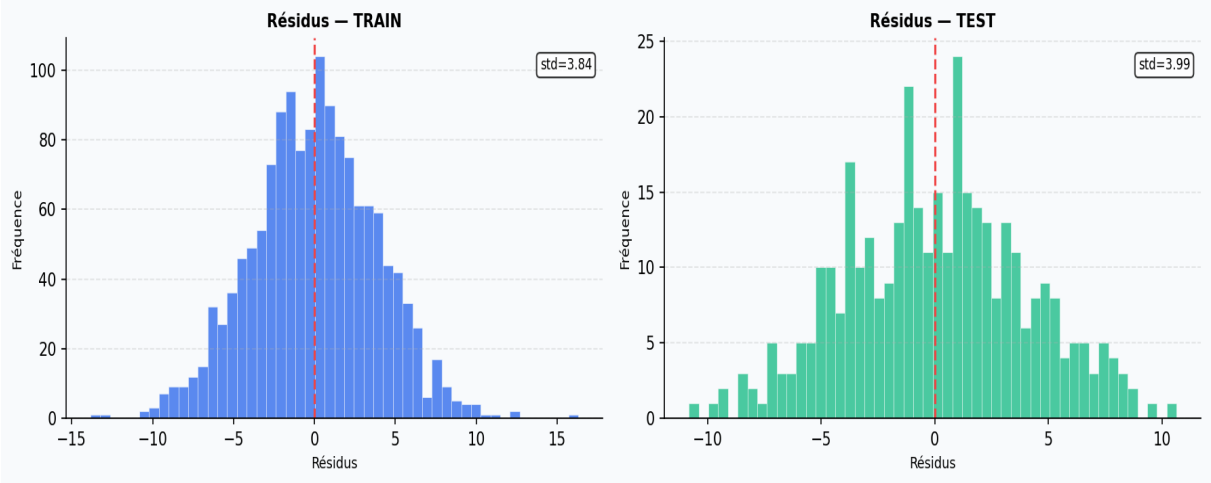

Figure 5 — Distribution des résidus sur train (bleu) et test (vert). Des distributions similaires indiquent une bonne généralisation — pas d'overfitting majeur.

7. Synthèse & Leçons Clés

- Corriger le look-ahead bias avec des lags temporels
- Construire 4 features : VIX(t-1), VIX(t-5), VIX(t-21), ΔVIX(t-1)
- Faire un train/test split chronologique (jamais aléatoire en time series)
- Entraîner le modèle uniquement sur le train set
- Évaluer avec R^2, RMSE et MAE sur le test set
- Comparer train vs test pour détecter l'overfitting
- Standardiser pour interpréter l'importance des features

Modules suivants

Module 3	Cross-validation temporelle (TimeSeriesSplit) — évaluation plus robuste
Module 4	Régularisation Ridge/Lasso — quand on a beaucoup de features
Module 5	Modèles non-linéaires — Random Forest, XGBoost
Module 6	Backtesting d'une stratégie de trading sur la volatilité

■ ***Question pour le Module 3 :*** *Le train/test split unique (80/20) a un défaut : le résultat dépend du choix de la date de coupure. Si on coupe en 2021 plutôt qu'en 2022, le R^2 test change. Comment évaluer un modèle de façon plus robuste, en utilisant toutes les données disponibles ?*